



Universidad Centroamericana José Simeón Cañas (UCA)

SERVICONECTA MOBILE

Documentación Técnica del Proyecto

Proyecto Final — Programación de Dispositivos Móviles
Ciclo 01-2026

Equipo de Desarrollo

Mario Alexander Molina Fuentes — 00372624

Miguel Alberto Escobar Estrada — 00401624

Ricardo Alberto Pineda Hernández — 00378824

Chelsea Sayuri Mejía Vásquez — 00164523

Carlos David Vásquez Martínez — 00063923

Contenido

- 1. Descripción General del Proyecto3
 - 1.1 Funcionalidades Principales (MVP)3
 - 1.2 Stack Tecnológico.....3
- 2. Arquitectura del Sistema4
 - 2.1 Componentes del Sistema.....4
 - 2.2 Flujo de Datos MVVM (App Android).....4
 - 2.3 Configuración del Backend4
 - 2.4 Interacción entre Componentes (Diagrama de Secuencia).....5
- 3. Base de Datos — ServiConectaDB.....6
 - 3.1 Diagrama Relacional (Descripción de Relaciones)6
 - 3.2 Diccionario de Datos6
- 4. Diagrama Relacional — Representación Visual8
- 5. Diccionario de APIs y Endpoints9
 - 5.1 Grupo: Autenticación (/auth)9
 - 5.2 Grupo: Público (/public) 11
- 6. Resumen de Endpoints — Tabla Rápida 14
- 7. Firebase Firestore — Mensajería en Tiempo Real 15
 - 7.1 Estructura de Colecciones 15
 - 7.2 Formato del chatId 15
 - 7.3 Modelo de Mensaje..... 15
 - 7.4 Operaciones Firestore en la App 15
- 8. Navegación de la App Android 16
 - 8.1 ViewModels..... 16
 - 8.2 Gestión de Sesión (SessionManager) 16
- 9. Modelos de Datos (DTOs) 18
- 10. Consideraciones de Seguridad 19
 - 10.1 Implementadas..... 19
 - 10.2 Pendientes / Recomendaciones 19
- 11. Guía de Configuración e Instalación 20
 - 11.1 Requisitos Previos 20
 - 11.2 Configuración del Backend 20
 - 11.3 Configuración de la App Android 20
- 12. Equipo de Desarrollo 20

1. Descripción General del Proyecto

ServiConecta Mobile es una plataforma Full-Stack nativa para Android diseñada para conectar a usuarios con profesionales técnicos en El Salvador. El proyecto representa la refactorización integral de una solución web previa, migrando toda la infraestructura a un ecosistema basado 100% en Kotlin.

1.1 Funcionalidades Principales (MVP)

- Navegación de Invitado: Exploración libre de categorías y perfiles profesionales sin registro obligatorio.
- Gestión de Perfiles: Los contratistas pueden configurar su biografía, años de experiencia y categoría de servicio.
- Chat en Tiempo Real: Comunicación bidireccional instantánea mediante Firebase Firestore.
- Sistema de Reputación: Calificaciones y reseñas dinámicas calculadas en tiempo real desde el servidor.
- Autenticación Segura: Registro y Login con roles diferenciados y seguridad mediante hashing de contraseñas (BCrypt).

1.2 Stack Tecnológico

Componente	Tecnologías
Lenguaje	Kotlin (Android y Backend)
Frontend Móvil	Jetpack Compose, Material Design 3, ViewModel
Networking	Retrofit 2, Gson, OkHttp
Backend	Ktor Framework (Kotlin Server), Netty Engine
BD Relacional	Microsoft SQL Server 2022
BD NoSQL	Firebase Firestore (mensajes en tiempo real)
Seguridad	BCrypt para hash de contraseñas
Arquitectura	MVVM (Model-View-ViewModel) + Patrón Repositorio

2. Arquitectura del Sistema

El sistema sigue el patrón MVVM y el Patrón Repositorio, con separación clara entre capa de UI, lógica de negocio, y acceso a datos. Existen dos subsistemas distintos que se comunican entre sí:

2.1 Componentes del Sistema

Subsistema	Tecnología	Responsabilidad
App Android	Kotlin + Compose	Interfaz de usuario, navegación, consumo de API
Backend Ktor	Kotlin + Ktor	API REST, lógica de negocio, acceso a SQL Server
SQL Server	T-SQL	Persistencia de usuarios, contratistas, categorías y reseñas
Firebase Firestore	NoSQL / Google	Mensajería en tiempo real entre usuarios

2.2 Flujo de Datos MVVM (App Android)

UI (Compose)	ViewModel	RetrofitClient	Backend Ktor
Renderiza estado, captura eventos del usuario	Mantiene estado con mutableStateOf, llama a la API	Realiza llamadas HTTP al backend	Procesa peticiones y devuelve JSON

2.3 Configuración del Backend

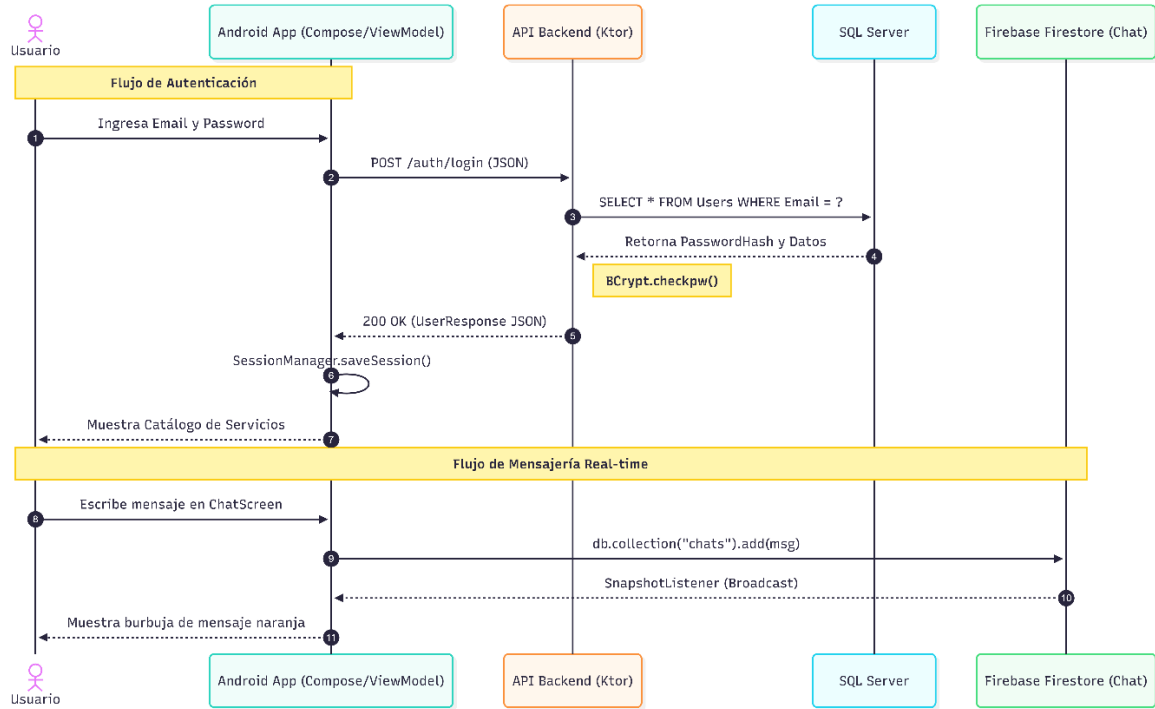
Servidor: `http://0.0.0.0:8080` (Puerto configurable en `application.yaml`)

Motor: Netty (via Ktor)

CORS: Habilitado para cualquier host (`anyHost()`) para desarrollo con emulador Android.

Serialización: `kotlinx.serialization` con formato JSON.

2.4 Interacción entre Componentes (Diagrama de Secuencia)



3. Base de Datos — ServiConectaDB

La base de datos relacional corre sobre Microsoft SQL Server. Contiene 5 tablas con sus respectivas restricciones, claves foráneas y relaciones.

3.1 Diagrama Relacional (Descripción de Relaciones)

Tabla Origen	Relación	Tabla Destino	Descripción
Users	1:1	ContractorProfiles	Un usuario contratista tiene exactamente un perfil profesional. ON DELETE CASCADE.
Users	1:N	Reviews (ClientID)	Un cliente puede dejar múltiples reseñas a distintos contratistas.
Users	1:N	Reviews (ContractorID)	Un contratista puede recibir múltiples reseñas de distintos clientes.
Users	M:N	Categories	Un contratista puede pertenecer a una o más categorías (vía ContractorCategories).
Categories	M:N	Users	Una categoría puede tener múltiples contratistas asociados.

3.2 Diccionario de Datos

Tabla: Users

Almacena todos los usuarios del sistema, tanto clientes como contratistas.

Campo	Tipo	PK	Nullable	Descripción
UserID	INT IDENTITY	Sí	No	Identificador único autoincremental del usuario.
Email	NVARCHAR(255)	No	No	Correo electrónico único del usuario (UNIQUE).
PasswordHash	NVARCHAR(255)	No	No	Hash BCrypt de la contraseña. Nunca texto plano.
FirstName	NVARCHAR(100)	No	No	Nombre del usuario.
LastName	NVARCHAR(100)	No	No	Apellido del usuario.
UserRole	NVARCHAR(50)	No	No	Rol del usuario: 'Cliente' o 'Contratista' (CHECK constraint).
CreatedAt	DATETIME2	No	Sí	Fecha y hora de registro. Default: GETDATE().

Tabla: Categories

Catálogo de categorías de servicios disponibles en la plataforma.

Campo	Tipo	PK	Nullable	Descripción
CategoryID	INT IDENTITY	Sí	No	Identificador único de la categoría.
CategoryName	NVARCHAR(100)	No	No	Nombre único de la categoría (Ej: Plomero, Electricista).
Description	NVARCHAR(500)	No	Sí	Descripción opcional de la categoría.

Tabla: ContractorProfiles

Perfil profesional extendido de los contratistas. Relación 1:1 con Users.

Campo	Tipo	PK/FK	Nullable	Descripción
UserID	INT	PK/FK	No	FK a Users(UserID). ON DELETE CASCADE.
BusinessName	NVARCHAR(150)	No	Sí	Nombre del negocio o emprendimiento del contratista.
PhoneNumber	NVARCHAR(20)	No	Sí	Número de teléfono de contacto.
Bio	NVARCHAR(MAX)	No	Sí	Biografía o descripción detallada del servicio.
YearsOfExperience	INT	No	No	Años de experiencia. Default: 0.

Tabla: ContractorCategories (Tabla de Unión M:N)

Resuelve la relación muchos-a-muchos entre contratistas y categorías.

Campo	Tipo	PK/FK	Nullable	Descripción
UserID	INT	PK/FK	No	FK a Users(UserID). Parte de PK compuesta.
CategoryID	INT	PK/FK	No	FK a Categories(CategoryID). Parte de PK compuesta.
PriceMin	DECIMAL(10,2)	No	Sí	Precio mínimo del servicio en esa categoría.
PriceMax	DECIMAL(10,2)	No	Sí	Precio máximo del servicio en esa categoría.

Tabla: Reviews

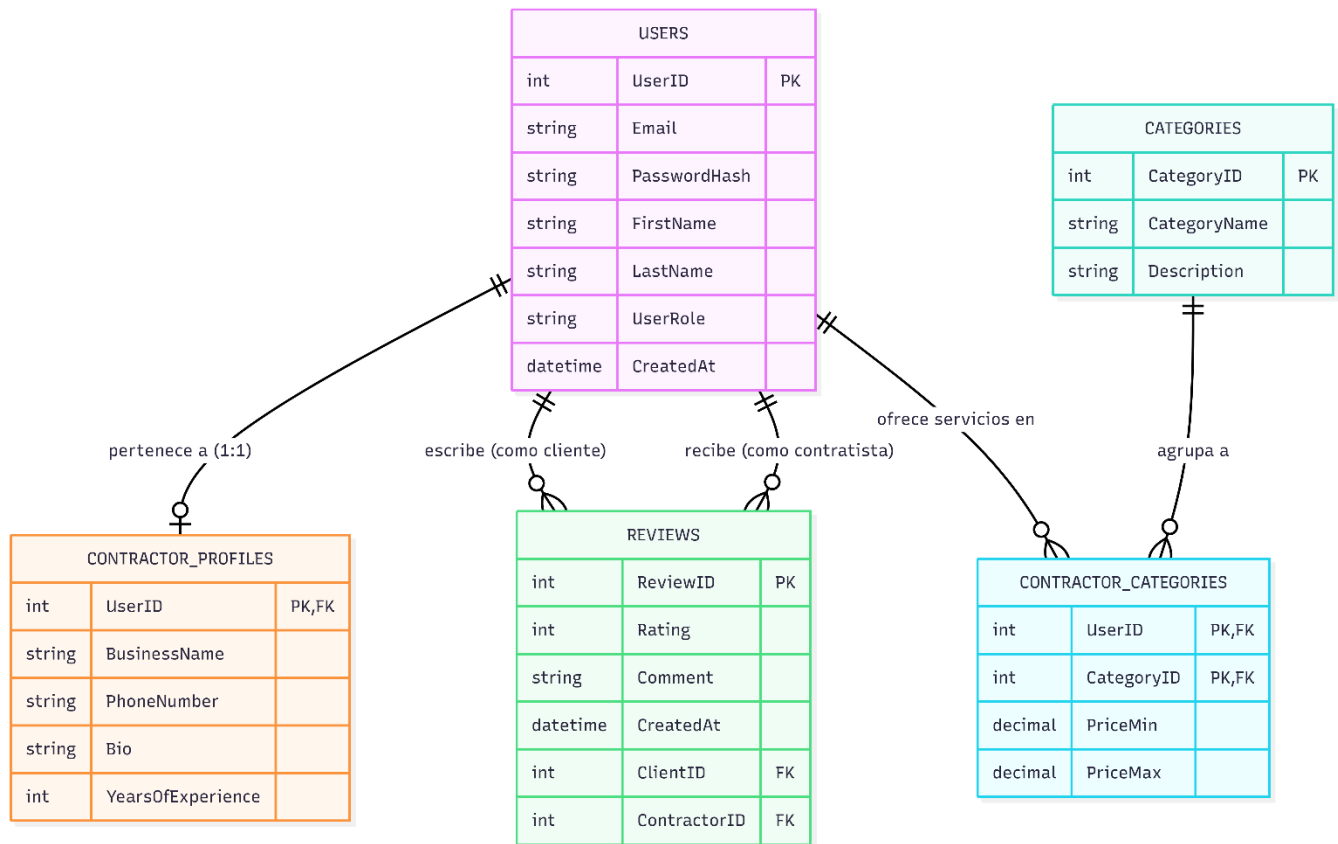
Almacena las calificaciones y comentarios de clientes hacia contratistas.

Campo	Tipo	PK/FK	Nullable	Descripción
ReviewID	INT IDENTITY	PK	No	Identificador único de la reseña.
Rating	INT	No	No	Calificación de 1 a 5 estrellas (CHECK constraint).
Comment	NVARCHAR(1000)	No	Sí	Comentario opcional del cliente.
CreatedAt	DATETIME2	No	Sí	Fecha de creación. Default: GETDATE().
ClientID	INT	FK	No	FK a Users(UserID). Quien escribe la reseña.
ContractorID	INT	FK	No	FK a Users(UserID). A quién va dirigida la reseña.

Restricción: CHK_NoSelfReview — El ClientID y ContractorID no pueden ser iguales (un usuario no puede calificarse a sí mismo).

4. Diagrama Relacional — Representación Visual

A continuación, se representa el modelo entidad-relación:



5. Diccionario de APIs y Endpoints

El backend Ktor expone una API REST sobre HTTP. Base URL: `http://<servidor>:8080/`

5.1 Grupo: Autenticación (/auth)

Endpoints que gestionan el ciclo de vida de los usuarios: registro, ingreso y actualización de perfil.

POST /auth/login

Autentica un usuario existente y retorna sus datos de sesión.

Método	POST
URL	/auth/login
Autenticación	No requerida (endpoint público)
Content-Type	application/json

Request Body (JSON):

Campo	Tipo	Descripción
Email	String	Correo electrónico del usuario.
Password	String	Contraseña en texto plano (se verifica contra el hash en DB).

Respuestas:

Código	Condición	Cuerpo de Respuesta
200 OK	Credenciales válidas	{ "UserID": 1, "Email": "...", "FirstName": "...", "LastName": "...", "UserRole": "Cliente" }
400 Bad Request	JSON mal formado	"Datos inválidos"
401 Unauthorized	Contraseña incorrecta	"Password incorrecta"
401 Unauthorized	Usuario no existe	"Usuario no encontrado"

POST /auth/register

Registra un nuevo usuario. Si el rol es 'Contratista', también crea automáticamente su ContractorProfile.

Método	POST
URL	/auth/register
Autenticación	No requerida (endpoint público)
Transacción	Sí — Usa <code>db.autoCommit = false</code> + <code>rollback</code> en caso de error

Request Body (JSON):

Campo	Tipo	Descripción
Email	String	Correo único del nuevo usuario.
Password	String	Contraseña en texto plano. Se hashea con BCrypt antes de guardar.
FirstName	String	Nombre del usuario.
LastName	String	Apellido del usuario.
UserRole	String	"Cliente" o "Contratista". Determina si se crea un ContractorProfile.

Respuestas:

Código	Condición	Cuerpo
201 Created	Registro exitoso	"Registrado"
400 Bad Request	JSON mal formado	"Datos inválidos"
500 Internal Server Error	Email duplicado u otro error de DB	Mensaje del error SQL

POST /auth/update-contractor

Actualiza el perfil profesional de un contratista y reasigna su categoría (operación DELETE+INSERT en ContractorCategories).

Método	POST
URL	/auth/update-contractor
Transacción	Sí — Usa db.autoCommit = false + rollback

Request Body (JSON):

Campo	Tipo	Descripción
UserID	Int	ID del contratista a actualizar.
BusinessName	String	Nombre del negocio.
Bio	String	Biografía o descripción del servicio.
YearsOfExperience	Int	Años de experiencia.
CategoryID	Int	ID de la categoría a la que se asocia el contratista.

Respuestas:

Código	Condición	Cuerpo
200 OK	Actualización exitosa	"Perfil actualizado"
500 Internal Server Error	Error de DB	Mensaje del error SQL

5.2 Grupo: Público (/public)

Endpoints de consulta de datos. No requieren autenticación. Disponibles para usuarios no registrados.

GET /public/categories

Retorna el listado completo de categorías de servicios disponibles en la plataforma.

Método	GET
URL	/public/categories
Parámetros	Ninguno
Autenticación	No requerida

Respuesta Exitosa 200 OK — Array de objetos Category:

Campo	Tipo	Descripción
CategoryID	Int	ID de la categoría.
CategoryName	String	Nombre de la categoría.
Description	String?	Descripción (puede ser null).

GET /public/contractors/{categoryId}

Retorna la lista de contratistas que pertenecen a una categoría específica.

Método	GET
URL	/public/contractors/{categoryId}
Parámetro de Ruta	categoryId (Int) — ID de la categoría a filtrar.
Autenticación	No requerida

JOIN ejecutado internamente:

```
Users INNER JOIN ContractorProfiles ON UserID
INNER JOIN ContractorCategories WHERE CategoryID = ?
```

Respuesta Exitosa 200 OK — Array de objetos Contractor:

Campo	Tipo	Descripción
UserID	Int	ID del contratista.
FirstName	String	Nombre.
LastName	String	Apellido.
BusinessName	String?	Nombre del negocio (puede ser null).
Bio	String?	Biografía (puede ser null).
YearsOfExperience	Int	Años de experiencia.

GET /public/contractor-detail/{id}

Retorna el perfil completo de un contratista: datos del perfil, lista de reseñas y calificación promedio calculada en el servidor.

Método	GET
URL	/public/contractor-detail/{id}
Parámetro de Ruta	id (Int) — ID del contratista.
Promedio	Calculado con COALESCE(AVG(CAST(Rating AS FLOAT)), 0.0) en SQL.

Respuesta Exitosa 200 OK — Objeto ContractorDetailResponse:

Campo	Tipo	Descripción
profile	Contractor	Objeto con datos del perfil del contratista.
reviews	List<Review>	Lista de reseñas ordenadas por fecha descendente.
averageRating	Double	Calificación promedio con 1 decimal (ej: 4.5).

Objeto Review dentro de la lista:

Campo	Tipo	Descripción
ReviewID	Int	ID de la reseña.
Rating	Int	Calificación de 1 a 5.
Comment	String?	Comentario (puede ser null).
ClientName	String	Nombre completo del cliente (FirstName + ' ' + LastName).
CreatedAt	String	Fecha en formato DD/MM/YYYY (CONVERT estilo 103).

Respuesta 404 Not Found: Cuando el ID no corresponde a ningún contratista registrado.

POST /public/reviews

Publica una nueva reseña de un cliente hacia un contratista.

Método	POST
URL	/public/reviews
Nota	No valida que ClientID sea diferente a ContractorID a nivel de API (la restricción existe a nivel de BD: CHK_NoSelfReview).

Request Body (JSON):

Campo	Tipo	Descripción
Rating	Int	Calificación de 1 a 5.
Comment	String	Comentario del cliente.
ClientID	Int	ID del usuario cliente que hace la reseña.
ContractorID	Int	ID del contratista calificado.

Respuestas:

Código	Condición	Cuerpo
201 Created	Reseña guardada	"Reseña guardada"
500 Internal Server Error	Error de DB (ej: CHK_NoSelfReview)	Mensaje del error SQL

6. Resumen de Endpoints — Tabla Rápida

Método	Endpoint	Auth	Descripción
POST	/auth/login	No	Autentica un usuario y retorna su información de sesión.
POST	/auth/register	No	Registra un nuevo usuario (Cliente o Contratista).
POST	/auth/update-contractor	No*	Actualiza perfil profesional y categoría del contratista.
GET	/public/categories	No	Lista todas las categorías de servicios disponibles.
GET	/public/contractors/{categoryId}	No	Lista contratistas filtrados por categoría.
GET	/public/contractor-detail/{id}	No	Detalle completo del contratista con reseñas y promedio.
POST	/public/reviews	No*	Publica una reseña de cliente hacia contratista.
GET	/	No	Health check: retorna 'ServiConecta Mobile Online!'

** Estos endpoints no tienen autenticación JWT implementada en el MVP. La identidad del usuario se valida implícitamente por el UserID en el cuerpo de la petición.*

7. Firebase Firestore — Mensajería en Tiempo Real

El sistema de chat no usa el backend Ktor sino Firebase Firestore directamente desde el cliente Android. Esto permite mensajería en tiempo real sin polling.

7.1 Estructura de Colecciones

Colección	Documento	Sub-Colección
chats	{chatId}	messages (lista de mensajes)

7.2 Formato del chatId

El ID del chat se construye combinando los dos UserIDs del backend, siempre en orden ascendente para garantizar que dos usuarios siempre compartan el mismo canal:

Caso	chatId generado
UserID=1 chatea con UserID=5	"1_5"
UserID=5 chatea con UserID=1	"1_5" (mismo canal)

7.3 Modelo de Mensaje

Campo	Tipo	Descripción
senderId	Int	UserID del usuario que envía el mensaje.
receiverId	Int	UserID del usuario destinatario.
message	String	Contenido del mensaje.
timestamp	Long	Marca de tiempo en milisegundos (System.currentTimeMillis()).
senderName	String	Nombre del remitente para mostrar en la UI.

7.4 Operaciones Firestore en la App

Operación	Descripción
addSnapshotListener	Escucha mensajes en tiempo real ordenados por timestamp ASC.
collection.add(msg)	Envía un nuevo mensaje al canal de chat.

8. Navegación de la App Android

La navegación usa Jetpack Navigation Compose con NavHost. La ruta inicial es 'catalog'.

Ruta	Pantalla	Acceso	Descripción
catalog	CatalogScreen	Público	Catálogo de categorías. Pantalla principal.
profile	ProfileScreen	Autenticado	Perfil del usuario. Formulario para contratistas.
contractors/{categoryId}	ContractorListScreen	Público	Lista de contratistas filtrada por categoría.
contractor_detail/{id}	ContractorDetailScreen	Público	Detalle del contratista + reseñas + calificación.
messages	MessagesScreen	Autenticado	Bandeja de chats del usuario.
chat/{receiverId}/{receiverName}	ChatScreen	Autenticado	Chat en tiempo real con Firebase Firestore.
login	LoginScreen	Público	Formulario de inicio de sesión.
register	RegisterScreen	Público	Formulario de registro de nuevo usuario.

8.1 ViewModels

ViewModel	Pantallas que lo usan	Estado que maneja
LoginViewModel	LoginScreen, RegisterScreen	email, password, firstName, lastName, selectedRole, isLoading, errorMessage, loginSuccess, registerSuccess
CatalogViewModel	CatalogScreen	categories: List<CategoryResponse>, isLoading
ContractorsViewModel	ContractorListScreen	contractors: List<ContractorResponse>, isLoading

8.2 Gestión de Sesión (SessionManager)

La sesión del usuario se persiste en SharedPreferences bajo el nombre 'ServiConectaPrefs'.

Método	Retorno	Descripción
saveSession(userId, firstName, userRole)	Unit	Guarda datos del usuario tras login exitoso.
isLoggedInIn()	Boolean	Verifica si hay sesión activa.
getUserName()	String	Retorna el nombre del usuario activo.
getUserRole()	String	Retorna el rol: 'Cliente' o 'Contratista'.
getUserId()	Int	Retorna el UserID del usuario activo.
logout()	Unit	Limpia todas las preferencias de sesión.

9. Modelos de Datos (DTOs)

A continuación se listan todos los Data Transfer Objects usados en la comunicación entre la app Android y el backend Ktor.

Clase	Capa	Dirección	Campos principales
LoginRequest	Android / Backend	Cliente → Servidor	Email, Password
RegisterRequest	Android / Backend	Cliente → Servidor	Email, Password, FirstName, LastName, UserRole
UpdateContractorRequest	Android / Backend	Cliente → Servidor	UserID, BusinessName, Bio, YearsOfExperience, CategoryID
ReviewRequest	Android / Backend	Cliente → Servidor	Rating, Comment, ClientID, ContractorID
UserResponse / Usuario	Android / Backend	Servidor → Cliente	UserID, Email, FirstName, LastName, UserRole
CategoryResponse / Category	Android / Backend	Servidor → Cliente	CategoryID, CategoryName, Description
ContractorResponse / Contractor	Android / Backend	Servidor → Cliente	UserID, FirstName, LastName, BusinessName, Bio, YearsOfExperience
ContractorDetailResponse	Android / Backend	Servidor → Cliente	profile: Contractor, reviews: List<Review>, averageRating: Double
ReviewResponse / Review	Android / Backend	Servidor → Cliente	ReviewID, Rating, Comment, ClientName, CreatedAt
Message	Android (Firebase)	Bidireccional	senderId, receiverId, message, timestamp, senderName
InboxChat	Android (Local)	Local	otherUserId, otherUserName, lastMessage, chatId

10. Consideraciones de Seguridad

10.1 Implementadas

- BCrypt para hash de contraseñas: Las contraseñas nunca se almacenan en texto plano. Se usa BCrypt.hashpw() en el registro y BCrypt.checkpw() en el login.
- Prepared Statements: Todos los queries usan PreparedStatement para prevenir inyección SQL.
- Transacciones: Las operaciones multi-tabla (registro de contratista, actualización de perfil) usan transacciones con rollback ante errores.
- CORS habilitado con anyHost(): Necesario para el desarrollo con emulador Android, pero debe restringirse en producción.
- Restricción de Roles a nivel de BD: UserRole tiene CHECK constraint ('Cliente' | 'Contratista').
- CHK_NoSelfReview: Restricción de BD que impide que un usuario se califique a sí mismo.

10.2 Pendientes / Recomendaciones

- JWT (JSON Web Tokens): Los endpoints autenticados (/auth/update-contractor, /public/reviews) no validan identidad a nivel de API. Se recomienda implementar tokens JWT en el Authorization header.
- HTTPS: El servidor corre sobre HTTP plano. En producción se recomienda usar HTTPS con certificado SSL.
- Rate Limiting: No hay límite de intentos de login. Se recomienda implementar para prevenir ataques de fuerza bruta.
- Validación de email: No hay validación de formato de email en el servidor.
- Credenciales de BD en código: DbConnection.kt contiene usuario y contraseña hardcodeados. Deben migrarse a variables de entorno.
- CORS restrictivo: Cambiar anyHost() a una lista blanca de orígenes permitidos en producción.

11. Guía de Configuración e Instalación

11.1 Requisitos Previos

Componente	Versión / Detalle
Android Studio	Ladybug o superior
IntelliJ IDEA	Para el Backend Ktor
JDK	17 o superior
SQL Server	Instancia con BD ServiConectaDB creada
Firebase	Proyecto configurado con Firestore. Archivo google-services.json en app/
Kotlin	2.2.10 (Android) / 2.3.21 (Backend)

11.2 Configuración del Backend

1. Abrir la carpeta ServiConectaMobileBackend en IntelliJ IDEA.
2. Configurar la cadena de conexión en src/main/kotlin/database/DbConnection.kt: cambiar URL, USER y PASS.
3. Ejecutar el script ServiConectaDB.sql en SQL Server para crear la base de datos y tablas.
4. Ejecutar la función main del proyecto. El servidor quedará activo en el puerto 8080.

11.3 Configuración de la App Android

5. Abrir la carpeta ServiConectaMobile en Android Studio.
6. Actualizar la IP del servidor en network/ApiService.kt: reemplazar BASE_URL con la IPv4 local de la máquina donde corre el backend.
7. Colocar el archivo google-services.json de Firebase en ServiConectaMobile/app/.
8. Sincronizar Gradle (Sync Project with Gradle Files).
9. Ejecutar en dispositivo físico o emulador Android.

12. Equipo de Desarrollo

Nombre	Caré	Rol
Mario Alexander Molina Fuentes	00372624	Desarrollador
Miguel Alberto Escobar Estrada	00401624	Desarrollador
Ricardo Alberto Pineda Hernández	00378824	Desarrollador
Chelsea Sayuri Mejía Vásquez	00164523	Desarrollador
Carlos David Vásquez Martínez	00063923	Desarrollador

Institución: Universidad Centroamericana José Simeón Cañas (UCA)

Materia: Programación de Dispositivos Móviles

Ciclo: 01-2026

Repositorio: <https://github.com/JustLex98/PDM-ProyectoFinal-ServiConecta.git>